

Master API Handbook ❖❖❖ Chapter 1: Foundations of APIs

- [The Master API Handbook](#)
 - [Chapter 1 — Module 1: Foundations of APIs](#)
 - [1. Definition](#)
 - [2. Why APIs Exist](#)
 - [3. Real World Analogy](#)
 - [4. Architecture Diagram](#)
 - [5. Step-by-Step Working \(What Actually Happens\)](#)
 - [6. Technical Deep Dive](#)
 - [7. Code Examples](#)
 - [8. Real Production Example](#)
 - [9. Best Practices](#)
 - [10. Common Mistakes](#)
 - [11. Performance Considerations](#)
 - [12. Security Considerations](#)
 - [13. Interview Questions](#)
 - [14. Revision Notes \(One-Page Summary\)](#)
 - [15. Key Points to Remember](#)
 - [Cheat Sheet](#)
 - [Practical Exercises](#)
 - [Mini Project](#)

The Master API Handbook

Chapter 1 — Module 1: Foundations of APIs

From Zero to Senior Backend Engineer

1. Definition

An **API** (Application Programming Interface) is a set of rules that allows one piece of software to ask another piece of software to do something, and to get an answer back — without either side needing to know how the other one works on the inside.

Think of “software talking to software.” Your phone’s weather app doesn’t have its own weather station. It sends a request to a weather company’s server: “give me today’s weather for Kolkata.” The server replies with data. The app never saw the weather company’s database, its code, or its servers — it just used the *interface* the weather company exposed.

A few more precise definitions we will use throughout this book:

- **Client** — the program that *asks* for something (a mobile app, a browser, another server).
- **Server** — the program that *has* the data or the ability to do the work, and *answers* requests.
- **Request** — the message the client sends (“please do X”).
- **Response** — the message the server sends back (“here is the result of X”).
- **Protocol** — an agreed set of rules both sides follow so the conversation makes sense (like a shared language).

Almost everything in this book is really about one question: **how do we design a reliable, safe, fast conversation between a client and a server?**

2. Why APIs Exist

Before APIs became standard, software systems were built as single, giant, tightly-glued programs (“monoliths” that were also *tightly coupled* — meaning if you changed one part, you risked breaking every other part). If a bank wanted its mobile app to check an account balance, the mobile team would often need to understand the exact internal database structure of the core banking system. That is fragile, slow to build, and dangerous — one small internal change could break dozens of dependent systems.

APIs solve four problems at once:

1. **Separation of concerns.** The team that builds the payments system doesn’t need to know how the mobile app renders buttons. They agree on a contract (the API), and each team builds independently behind that contract.
2. **Reusability.** One backend API can serve a web app, an iOS app, an Android app, a smartwatch app, and a partner company’s system — all at the same time, without rewriting the core logic five times.
3. **Security boundary.** The API only exposes what it chooses to expose. The database, internal logic, secrets, and infrastructure stay hidden behind it.
4. **Scalability of teams, not just servers.** In a large company like Amazon, thousands of engineers work on thousands of services. APIs are the “contracts” that let all of them move fast without stepping on each other’s code.

In short: **APIs exist so that software can be built, changed, and scaled by many independent teams and systems, without everyone needing to know everyone else’s internals.**

3. Real World Analogy

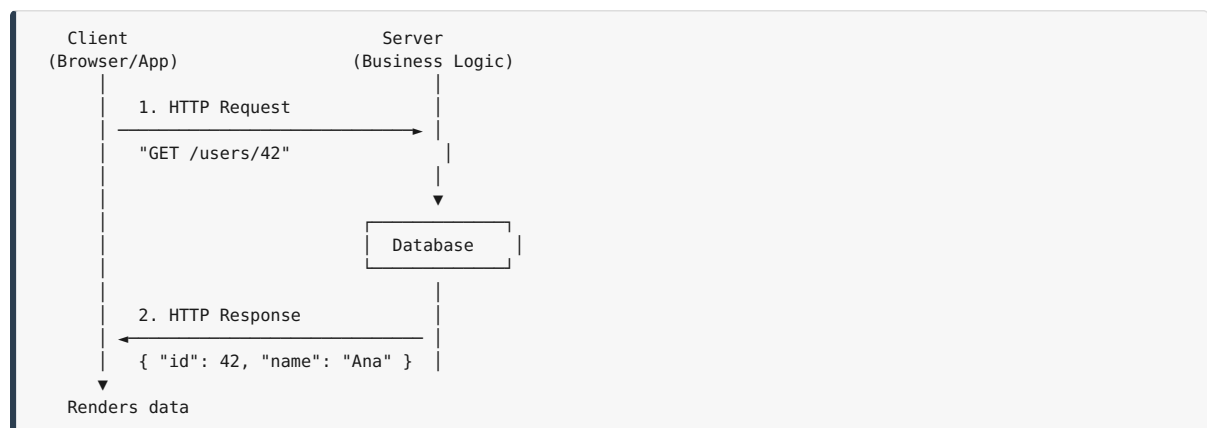
The Restaurant Analogy (the most important one — we will reuse it constantly):

- **You (the customer)** = the Client.
- **The Kitchen** = the Server (where the real work — cooking — happens).
- **The Menu** = the API documentation (it tells you exactly what you’re allowed to ask for, and what format to ask in).
- **The Waiter** = the API itself (the interface). You never walk into the kitchen yourself. You tell the waiter what you want using the menu’s language (“One Margherita Pizza, medium spicy”), the waiter carries that request to the kitchen, the kitchen prepares it, and the waiter brings back the response (your food).
- **You don’t need to know** how the chef cooks, what oven is used, or how the kitchen is organized. That’s hidden from you. You only interact through the fixed interface: the menu and the waiter.
- If the kitchen changes its recipe internally (say, switches suppliers) but the final dish tastes and looks the same, **you never notice** — because the *interface* (the menu item and how it’s delivered) didn’t change. This is the essence of good API design: internals can change freely as long as the interface (the “contract”) stays stable.

The Bank Analogy (for authentication concepts later): You don’t get to walk behind the counter and open the vault yourself. You show ID (authentication — proving who you are), and the teller checks if you’re *allowed* to withdraw this specific amount from this specific account (authorization — checking what you’re permitted to do). We’ll return to this in Module 7.

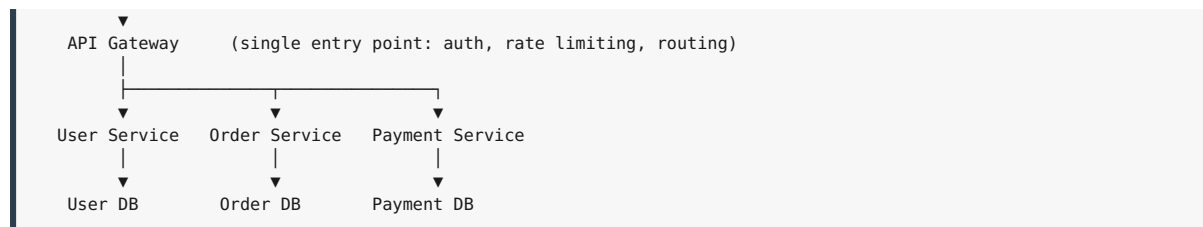
4. Architecture Diagram

The simplest possible API interaction:



A more realistic production setup, which we will build toward across this book:





Don't worry about Gateways, Load Balancers or microservices yet — we introduce each piece as its own chapter (Modules 9 and 13). For now, just anchor the core idea: **Client sends a request → some server-side system processes it → a response comes back.**

5. Step-by-Step Working (What Actually Happens)

Let's trace one real example end-to-end: you open a shopping app and it shows your order history. Here is exactly what happens, step by step, when the app calls `GET /orders` :

```
1. App constructs a request
↓
2. DNS lookup: "api.shop.com" → IP address (e.g., 52.14.x.x)
↓
3. TCP connection established (3-way handshake) between device and server
↓
4. TLS handshake (if HTTPS) – encryption keys are exchanged
↓
5. HTTP request sent over that secure connection:
  GET /orders HTTP/1.1
  Host: api.shop.com
  Authorization: Bearer <token>
↓
6. Server receives the request
↓
7. Authentication check – "Who is this?" (valid token?)
↓
8. Authorization check – "Is this user allowed to see these orders?"
↓
9. Business logic runs – fetch orders belonging to this user
↓
10. Database query executed
↓
11. Data formatted into JSON
↓
12. HTTP response sent back:
  HTTP/1.1 200 OK
  Content-Type: application/json
  [ {...order1...}, {...order2...} ]
↓
13. App receives response, parses JSON, renders the order list on screen
```

Every single one of these 13 steps matters, and later chapters go deep into each: TCP (this chapter), TLS/HTTPS (this chapter), Authentication/Authorization (Module 7), status codes (Module 4), JSON structure and response design (Module 5).

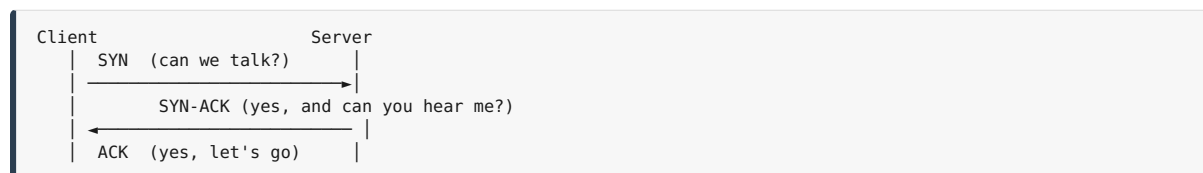
6. Technical Deep Dive

This is the heart of Module 1. We'll build up from the network layer to the REST style layer.

6.1 TCP — The Reliable Pipe

Before any API conversation can happen, two computers need a reliable communication channel. That's **TCP (Transmission Control Protocol)**. TCP guarantees:

- **Data arrives in order** (packet 1 before packet 2, even if they took different paths across the internet).
- **No data is lost silently** (if a packet is missing, it's re-sent).
- **A connection is established first** via the famous **3-way handshake**:



✓ Connection Established

Only after this handshake does any actual API data flow. This is also why the very first request to a new server always feels a little slower than the following ones — the connection is often reused (kept alive) afterward.

6.2 HTTP — The Language of APIs

HTTP (HyperText Transfer Protocol) is the language spoken *on top of* that TCP pipe. It defines the format of a request and a response, so any client and any server — written in any programming language, by any company — can understand each other.

An HTTP request has this shape:

```
GET /users/42 HTTP/1.1      ← Method + Path + Version
Host: api.company.com      ← Headers (metadata)
Authorization: Bearer abc123
Accept: application/json

(optional body — used for POST/PUT/PATCH)
```

An HTTP response has this shape:

```
HTTP/1.1 200 OK            ← Version + Status Code + Status Text
Content-Type: application/json ← Headers
Content-Length: 128

{ "id": 42, "name": "Ana" } ← Body
```

We cover **methods** in Module 2, **status codes** in Module 4, and **headers/body** in Module 3 in full depth. For now, understand: HTTP is just a **standardized envelope format** for requests and responses, sent over a TCP connection.

6.3 HTTPS — HTTP Plus Encryption

HTTPS is HTTP running inside a **TLS (Transport Layer Security)** encrypted tunnel. Without HTTPS, anyone sitting on the same network (a coffee shop Wi-Fi, an ISP, an attacker on the path) could read — or even modify — your API traffic in plain text, including passwords and tokens. This is called a **Man-in-the-Middle (MITM) attack**.

```
Without HTTPS:
Client —[ "password":"12345" in plain text ]→ Attacker can read it → Server

With HTTPS:
Client —[ 8f3a9c2e1... encrypted gibberish ]→ Attacker sees nothing useful → Server
```

TLS also does something equally important: **it proves the server is who it claims to be**, using digital certificates issued by trusted Certificate Authorities (CAs). This prevents an attacker from pretending to be `api.yourbank.com`.

In 2026, HTTPS is not optional for any serious API — browsers actively warn users away from plain HTTP, and most modern platforms (App Store, Play Store, browsers) simply refuse to allow plain HTTP traffic by default.

6.4 JSON — The Data Format

Most modern APIs exchange data as **JSON (JavaScript Object Notation)** — a lightweight, human-readable text format:

```
{
  "id": 42,
  "name": "Ana",
  "isActive": true,
  "roles": ["admin", "editor"],
  "address": { "city": "Kolkata", "pincode": "700001" }
}
```

JSON won over older formats like XML because it's smaller, easier to read, and maps naturally onto objects/dictionaries in almost every programming language. We use JSON in every code example in this book.

6.5 REST — A Style, Not a Protocol

REST (Representational State Transfer) is not a technology or a library — it is an **architectural style**: a set of design principles for building APIs over HTTP, described by Roy Fielding in his year-2000 doctoral dissertation. A REST API is simply an HTTP API that follows these principles well.

6.6 The REST Constraints (in simple English)

Constraint	Simple Meaning
Client-Server	Client and server are separate; the client doesn't need to know how the server is implemented, and vice versa.
Statelessness	Every request must contain everything the server needs to understand it. The server does not remember anything about you between requests (no "session memory" on the server tied to a connection). Each request stands alone.
Cacheability	Responses must clearly say whether they can be stored/reused later (cached) to avoid repeating expensive work.
Uniform Interface	Every resource is accessed the same predictable way — same URL patterns, same standard HTTP methods, same conventions everywhere in the API.
Layered System	The client can't tell (and shouldn't need to know) whether it's talking directly to the server or through several layers (gateway, load balancer, cache) in between.
Code on Demand (optional)	The server can occasionally send executable logic to the client (rare in typical APIs — mostly a web browser/JavaScript concept).

Statelessness deserves extra attention, because it's the constraint beginners misunderstand most. It does *not* mean "the app has no data." It means: the *server* doesn't keep a memory of "where you are" in a conversation. Every request must carry its own identity proof (like a token), so any server in a cluster of a thousand servers could handle it — not just the one you talked to last time. This is *exactly* what makes REST APIs easy to scale horizontally (add more servers).

6.7 CRUD — The Four Operations on Data

Almost everything an API does boils down to four operations on data, known as **CRUD**:

Letter	Operation	Meaning
C	Create	Add new data (e.g., create a new user)
R	Read	Fetch existing data
U	Update	Modify existing data
D	Delete	Remove data

In Module 2, we'll map CRUD directly onto HTTP methods (`POST` →Create, `GET` →Read, `PUT` / `PATCH` →Update, `DELETE` →Delete).

6.8 Resources and Endpoints

A **resource** is any "thing" your API exposes — a user, an order, a product, an invoice. REST thinks in terms of **nouns**, not verbs. You don't design an endpoint called `/getUserData`; instead the *resource* is `/users`, and the HTTP *method* (`GET`) tells you what action to perform on it.

An **endpoint** is a specific URL where a resource (or a collection of resources) can be accessed:

```

/users          → the collection of all users
/users/42       → one specific user (id = 42)
/users/42/orders → the collection of orders belonging to user 42

```

This naming style ("nested, plural, noun-based") is the standard we teach in full in Module 5.

6.9 Client-Server Architecture in Practice

Zooming out, "client-server" is the foundational shape almost every application follows:



The client owns *presentation* (how things look, how the user interacts). The server owns *truth* (the actual data, the actual rules, security enforcement). This separation is why you can never trust data validation done only in the client app — a malicious user can bypass your app entirely and call your API directly with a tool like `curl` or Postman. Real validation and security *must* live on the server. We revisit this constantly in Module 8 (Security).

7. Code Examples

Let's build the smallest possible real API in three different stacks, all doing the same thing: GET `/users/:id` returning a user's data as JSON.

7.1 Express.js (TypeScript)

```
// server.ts
import express, { Request, Response } from "express";

const app = express();
app.use(express.json());

// In-memory "database" for this simple example
const users = [{ id: 42, name: "Ana", email: "ana@example.com" }];

app.get("/users/:id", (req: Request, res: Response) => {
  const userId = Number(req.params.id);
  const user = users.find((u) => u.id === userId);

  if (!user) {
    return res.status(404).json({ error: "User not found" });
  }

  return res.status(200).json(user);
});

app.listen(3000, () => {
  console.log("API running on http://localhost:3000");
});
```

7.2 NestJS (TypeScript)

NestJS organizes code into **Controllers** (handle HTTP), **Services** (business logic), and **Modules** (grouping) — a clean architecture pattern used heavily in production backend systems.

```
// users.controller.ts
import { Controller, Get, Param, NotFoundException } from "@nestjs/common";
import { UsersService } from "./users.service";

@Controller("users")
export class UsersController {
  constructor(private readonly usersService: UsersService) {}

  @Get(":id")
  findOne(@Param("id") id: string) {
    const user = this.usersService.findById(Number(id));
    if (!user) {
      throw new NotFoundException("User not found");
    }
    return user;
  }
}
```

```
// users.service.ts
import { Injectable } from "@nestjs/common";

interface User {
  id: number;
  name: string;
  email: string;
}

@Injectable()
export class UsersService {
  private readonly users: User[] = [
    { id: 42, name: "Ana", email: "ana@example.com" },
  ];

  findById(id: number): User | undefined {
    return this.users.find((u) => u.id === id);
  }
}
```

7.3 Next.js App Router (Route Handler, TypeScript)

```
// app/api/users/[id]/route.ts
```

```
import { NextRequest, NextResponse } from "next/server";

const users = [{ id: 42, name: "Ana", email: "ana@example.com" }];

export async function GET(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  const userId = Number(params.id);
  const user = users.find((u) => u.id === userId);

  if (!user) {
    return NextResponse.json({ error: "User not found" }, { status: 404 });
  }

  return NextResponse.json(user, { status: 200 });
}
```

Notice all three examples do the *exact same conceptual thing*: receive an HTTP GET request with a path parameter, look up a resource, return JSON with the right status code. Different frameworks, same underlying HTTP/REST principles — that consistency is the entire point of learning the fundamentals well before the frameworks.

8. Real Production Example

- **Stripe** built its entire business on API design quality. Stripe’s public API is famous for being extremely consistent, predictable, and well-documented — every resource follows the same URL and response shape conventions, which is a huge reason developers trust it enough to route real money through it.
- **Netflix** exposes internal APIs between hundreds of microservices (recommendations, billing, playback, etc.). Each team owns their service behind a stable API contract, so the “Recommendations team” can completely rewrite their internal recommendation algorithm without ever notifying the “Playback team” — because the API interface didn’t change.
- **Amazon** famously mandated (internally, in the early 2000s, under Jeff Bezos) that every team must expose their functionality only through well-defined APIs, with no back-door database access between teams — this single decision is widely credited as the foundation that made AWS (Amazon Web Services) possible years later, since services were already built as independently deployable, API-driven units.
- **Uber** relies on stateless REST-style APIs so that any of thousands of backend servers can handle any rider’s request, because no server needs to “remember” which server you talked to last — critical for a system with millions of concurrent trips worldwide.

9. Best Practices

- Always design the **API contract first** (what endpoints, what data shapes) before writing implementation code — this is called **API-first design**.
- Keep the API **stateless** — never store per-user session data only in server memory tied to one machine.
- Use **HTTPS everywhere**, with no exceptions, including internal service-to-service calls where possible.
- Treat resources as **nouns**, actions as **HTTP methods** — don’t invent verb-based endpoints.
- Version your API from day one (we cover this fully in Module 5) so you can evolve without breaking existing clients.
- Keep responses **predictable and consistent** across the whole API (same date formats, same error shape, same pagination style everywhere).

10. Common Mistakes

1. **Putting verbs in URLs** — e.g., `/getUser`, `/createOrder`. This fights against REST’s resource-based design and quickly becomes inconsistent as the API grows. *Fix*: `GET /users/:id`, `POST /orders`.
2. **Storing session state on a single server** — e.g., keeping the logged-in user’s cart in that one server’s memory. This breaks the moment you add a second server or that server restarts. *Fix*: store state in the client (token) or in a shared store (database/Redis), not server memory.
3. **Trusting client-side validation as security** — a beginner validates the form in the browser and assumes that’s enough. Anyone can bypass the browser and call the API directly. *Fix*: always revalidate everything on the server.
4. **Confusing HTTP with HTTPS as if they were interchangeable in production** — using plain HTTP “just for now” and forgetting to switch before launch. *Fix*: HTTPS from the very first commit, even in development, using tools like local certificates.
5. **Not understanding statelessness**, and trying to build “the server remembers which page you were on” logic — leads to bugs the moment there’s more than one server instance. *Fix*: the client always sends full context (like a token and any needed parameters) with every request.

11. Performance Considerations

- **Statelessness enables horizontal scaling** — because no server holds session memory, you can add 10, 100, or 1000 identical servers behind a load balancer, and any of them can serve any request. This is *why* REST-style stateless APIs scale so well for companies like Netflix and Uber.
 - **TCP connection reuse** (keep-alive) avoids repeating the 3-way handshake (and TLS handshake) for every single request, which meaningfully reduces latency — this is handled automatically by most HTTP clients and servers today, but it’s worth understanding why the *first* request on a fresh connection is slower.
 - **Payload size matters.** JSON is readable but verbose. For very high-throughput internal services, some companies use more compact binary protocols (we compare this to gRPC in Module 10).
 - **Network distance (latency) matters more than raw processing speed** for most APIs — a server in Mumbai answering a request from Kolkata will almost always be faster than a technically “more optimized” server sitting in the US, purely due to the physical distance data has to travel. This is why companies use **CDNs** and **multi-region deployments**.
-

12. Security Considerations

- **Always use HTTPS** to prevent Man-in-the-Middle attacks (see section 6.3).
 - **Never trust the client.** Every piece of data coming from the client — including things like user IDs in the request — must be re-verified on the server. A client claiming “I am user 42” must *prove* it (we cover this fully in Module 7 — Authentication).
 - **Don’t leak internal details in errors.** A stack trace or raw database error message returned to the client can reveal your internal architecture to an attacker. Return clean, generic error messages instead (Module 4 and Module 5 cover proper error response design).
 - **Statelessness has a security upside too:** because no sensitive session state sits in server memory across requests, there’s less risk of one compromised server leaking another user’s live session — as long as tokens are handled correctly (Module 7).
-

13. Interview Questions

Basic

- **Q: What is an API?** A: A set of rules that lets one piece of software request functionality or data from another piece of software, without either side needing to know the other’s internal implementation.
- **Q: What is the difference between HTTP and HTTPS?** A: HTTPS is HTTP transmitted inside an encrypted TLS tunnel. It adds encryption (so data can’t be read in transit) and server identity verification (so you know you’re really talking to the real server), while HTTP sends everything in plain, readable text.
- **Q: What does CRUD stand for?** A: Create, Read, Update, Delete — the four fundamental operations performed on data in almost every application.

Intermediate

- **Q: What does “statelessness” mean in REST, and why does it matter?** A: It means the server does not store any memory of a client’s previous requests; every request must carry all the information needed to process it (e.g., an auth token). This matters because it allows *any* server in a cluster to handle *any* request, which is essential for horizontal scaling and high availability.
- **Q: What is the difference between a resource and an endpoint?** A: A resource is the conceptual “thing” being represented (e.g., a user). An endpoint is the specific URL used to access that resource (e.g., `/users/42`).
- **Q: Why does REST recommend nouns in URLs instead of verbs?** A: Because the HTTP method (GET, POST, PUT, DELETE) already expresses the action. Combining a verb in the URL *and* an HTTP method (`POST /createUser`) is redundant and breaks the uniform, predictable interface REST aims for.

Advanced

- **Q: Is a REST API required to use JSON?** A: No. REST as an architectural style doesn’t mandate a specific data format — early REST APIs commonly used XML. JSON became the de facto standard because of its simplicity and native fit with JavaScript/web clients, but a REST API could technically respond in XML, YAML, or any other format, based on content negotiation (the `Accept` header).
- **Q: Why is Amazon’s internal “API mandate” (circa 2002) considered historically significant for the creation of AWS?** A: By forcing every internal team to expose functionality only through well-defined service APIs (with no direct database access between teams), Amazon effectively built its infrastructure as a collection of independently deployable services years before “microservices” was a common term. Because these services were already built to be accessed externally through clean interfaces, it became feasible to expose some of them as commercial products — which became AWS.

- **Q: Explain the layered system constraint in REST and why it matters for real production systems.** A: The layered system constraint says a client cannot tell (and shouldn't need to care) whether it is talking directly to the origin server, or through intermediate layers like a load balancer, API gateway, or cache. This matters because it allows companies to insert new infrastructure — caching layers, gateways, security layers — transparently, without requiring any client to change, since the client-facing contract never changes.

14. Revision Notes (One-Page Summary)

- **API** = a defined way for software to talk to other software.
- **Client** asks, **Server** answers. **Protocol** = shared rules for the conversation.
- APIs exist to decouple teams/systems, enable reuse, and create a security boundary.
- **TCP** = the reliable connection layer underneath HTTP (3-way handshake).
- **HTTP** = the standard request/response envelope format for APIs.
- **HTTPS** = HTTP + TLS encryption + server identity verification. Always use it.
- **JSON** = the standard lightweight data format for modern APIs.
- **REST** = an architectural style (not a protocol) with 6 constraints: Client-Server, Statelessness, Cacheability, Uniform Interface, Layered System, Code on Demand (optional).
- **Statelessness** = every request is self-contained; server holds no per-client memory between requests. This is *the* key to horizontal scalability.
- **CRUD** = Create, Read, Update, Delete — the four fundamental data operations.
- **Resource** = a “thing” (noun) the API exposes. **Endpoint** = a specific URL to access it.
- URLs should be noun-based (/users), never verb-based (/getUsers).

15. Key Points to Remember

1. An API is a *contract*, not an implementation — internals can change freely as long as the contract holds.
2. Statelessness is the single most important idea behind REST's scalability story.
3. HTTPS is non-negotiable in any real system.
4. Never trust the client — all real validation and security enforcement happens on the server.
5. Resources are nouns; HTTP methods are verbs. Don't mix the two in your URL design.

Cheat Sheet

Term	One-Line Meaning
API	Software-to-software interface
Client	The requester
Server	The responder
TCP	Reliable connection layer (3-way handshake)
HTTP	Standard request/response format
HTTPS	HTTP + encryption + identity verification
JSON	Lightweight data exchange format
REST	Architectural style for HTTP APIs
Statelessness	Server remembers nothing between requests
CRUD	Create, Read, Update, Delete
Resource	A “thing” the API represents
Endpoint	The URL to access a resource

Practical Exercises

1. Use your browser's DevTools (Network tab) to open any website, and find one real HTTP request. Identify: the method, the URL, the status code, and at least two headers.
2. Write down, in your own words, why storing a shopping cart in a single server's memory would break if that company

grew to use 50 servers behind a load balancer.

3. Take an existing endpoint you've written at work (e.g., in Invorush) and check: is it named as a noun? Does it follow REST resource-naming? Rewrite it if not.
4. Using `curl`, manually send a GET request to any public API (e.g., `curl https://api.github.com/users/octocat`) and inspect the raw JSON response.

Mini Project

Build a tiny stateless “Book Catalog” API with one endpoint: `GET /books/:id`, returning a book's title, author, and year from an in-memory array — in **all three** stacks shown in this chapter (Express, NestJS, Next.js App Router). Confirm all three return identical JSON for the same request, and test what happens (status code + body) when you request a book ID that doesn't exist.

End of Chapter 1. Next: Chapter 2 — Module 2: HTTP Methods, Idempotency & Safe Methods.